



Cloud Native Application Delivery

GitOps Principles

 Copy page

This article discusses the principles of GitOps and the establishment of a governing body to standardize practices within the GitOps community.

When a new technology emerges, debates about its governance and the underlying principles essential for its responsible use are inevitable. A dedicated governing body helps set standards and ensures that the technology is developed and maintained in an unbiased, community-driven manner. For instance, Kubernetes—a leading container orchestration system—is guided by the CloudNative Computing Foundation (CNCF), which maintains its vendor-neutral and community-focused ethos.

This framework is particularly vital for modern approaches such as GitOps, a recent methodology for software delivery and deployment. To ensure clarity and consistency in GitOps practices, the establishment of a governing body is crucial. Recognizing this need, the CNCF formed the GitOps Working Group under its App Delivery Special Interest Group (SIG). This group is committed to defining a vendor-neutral understanding of GitOps while establishing a common language for the community.

The GitOps Working Group has initiated the OpenGitOps project, uniting experts and stakeholders within the GitOps community to standardize best practices.

>



Open GitOps Project



GitOps Working Group

This project represents a foundational step towards fostering collaboration and interoperability among the open source community, vendors, and end-user organizations.

GitOps Working Group



After extensive discussions, the OpenGitOps project defined GitOps through four fundamental principles:



>

outline crucial settings—such as container images, replica counts, service types, and additional configuration details—to directly describe the system's intended state.

2. The definitive desired state is maintained in Git

The definitive desired state is maintained in Git. After committing the desired configuration in a YAML file to Git, it becomes version-controlled, allowing change tracking over time and easy reversion to previous versions if needed. Furthermore, once applied to the cluster, the YAML file is treated as immutable; updates are applied by creating a new version rather than modifying the current one.

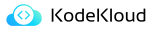
3. Approved changes are automatically propagated to the system

Approved changes are automatically propagated to the system. Tools like Flux or Argo CD continuously monitor the Git repository for updates. When changes are pushed, these tools detect the new configuration and automatically apply it to the Kubernetes cluster. Consider the following YAML configuration for a deployment:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
```



This automation minimizes manual intervention, thereby accelerating deployments, enhancing reliability, and reducing errors.



>

number of replicas changes manually—the GitOps tool promptly takes corrective action to restore the intended configuration.

By adhering to these four core principles, organizations can maintain a consistent and reliable application state while boosting operational efficiency and overall system integrity.



Watch Video

[Learn more >](#)

Was this page helpful?



Yes



No

What is GitOps

[< Previous](#)

Push vs Pull based Deployments

[Next >](#)

Powered by **mintlify**